Lecture 5 - Main Memory & Virtual Memory

1. Main Memory (RAM)

Main memory, often referred to as RAM (Random Access Memory), is a crucial component of any computer system. It serves as the primary storage location for data and instructions that the CPU actively uses. Its defining characteristic is "random access," meaning any byte of memory can be accessed directly in roughly the same amount of time, regardless of its physical location. This is in contrast to sequential access storage like hard drives or tape drives.

1.1 Types of RAM

There are two primary types of RAM commonly encountered:

- **SRAM (Static RAM):**
  - **Construction:** SRAM uses latches (typically 6 transistors for a single bit) to store each bit of data. This means it does not need to be constantly refreshed.
  - **Speed:** Much faster than DRAM due to its static nature.
  - **Cost:** More expensive to manufacture per bit due to its higher transistor count.
  - **Power Consumption:** Lower power consumption in standby mode.
  - **Usage:** Primarily used for cache memory (CPU caches like L1, L2, L3) where speed is paramount and the amount of memory needed is relatively small.
- **DRAM (Dynamic RAM):**
  - **Construction:** DRAM stores each bit of data in a capacitor, which requires a single transistor. Capacitors naturally discharge over time, so DRAM needs to be "refreshed" periodically (thousands of times per second) to maintain its data. This refresh process is what makes it "dynamic."
  - **Speed:** Slower than SRAM due to the refresh cycles.
  - **Cost:** Much cheaper to manufacture per bit due to its simpler structure.
  - **Power Consumption:** Higher power consumption due to constant refreshing.
  - **Usage:** The dominant form of main memory in personal computers, servers, and most other computing devices where large capacities are required at an affordable price.

1.2 Memory Addressing and Organization

- **Memory Addressing:**
  - Every byte in main memory has a unique numerical address.
  - The CPU uses these addresses to locate and retrieve specific data or instructions.
  - Addresses are typically represented in hexadecimal format (e.g., 0x0000FFFF).

- **Address Bus:** The CPU communicates memory addresses via the address bus. The width of the address bus determines the maximum amount of physical memory that can be addressed (e.g., a 32-bit address bus can address 232 bytes or 4 GB of memory).
- **Data Bus:** Data is transferred between the CPU and memory via the data bus. The width of the data bus determines how many bits can be transferred at once (e.g., a 64-bit data bus can transfer 8 bytes simultaneously).

- **Memory Organization:**
  - Memory is logically organized as a contiguous array of bytes.
  - However, physically, memory is often arranged in modules (e.g., DIMMs - Dual In-line Memory Modules) containing multiple RAM chips.
  - **Word Size:** Processors typically access memory in "words" rather than individual bytes. A word is a fixed-size group of bytes, often matching the processor's register size (e.g., 32-bit or 64-bit words).
  - **Memory Controller:** A dedicated hardware component (often integrated into the CPU or chipset) manages all memory access operations, including addressing, data transfer, and refresh cycles for DRAM.

1.3 Error Detection and Correction

Memory errors can occur due to various reasons, including electrical interference, manufacturing defects, or cosmic rays. Detecting and correcting these errors is crucial for system stability and data integrity.

- **Parity:**
  - **Concept:** A simple error detection scheme. An extra "parity bit" is added to a block of data.
  - **Even Parity:** The parity bit is set so that the total number of '1's in the data block (including the parity bit) is even.
  - **Odd Parity:** The parity bit is set so that the total number of '1's in the data block (including the parity bit) is odd.
  - **Detection:** If, upon reading, the parity rule is violated, an error is detected.
  - **Limitation:** Parity can only detect an odd number of bit errors (typically a single bit error). It cannot correct errors, nor can it detect an even number of bit errors.
- **CRC (Cyclic Redundancy Check) - Brief Mention:**
  - **Concept:** A more robust error detection method, widely used in data transmission and storage.

  - **Mechanism:** A mathematical algorithm generates a short, fixed-length "checksum" for a block of data. This checksum is appended to the data.
  - **Detection:** Upon receiving/reading, the receiver re-calculates the CRC. If the calculated CRC doesn't match the received CRC, an error is detected.
  - **Strength:** Highly effective at detecting various types of errors, including multiple bit errors, burst errors, etc.
  - **Limitation:** Primarily an error *detection* mechanism; while some CRC variants can be used for error correction, it's generally more complex than simple detection.

- **ECC (Error-Correcting Code) Memory:**
  - **Concept:** Utilizes more sophisticated algorithms (e.g., Hamming codes) to not only detect errors but also *correct* them.
  - **Mechanism:** Adds several extra bits (redundancy) to each word of data. These extra bits contain enough information to pinpoint and flip a single incorrect bit, or detect multiple incorrect bits.
  - **Usage:** Common in servers, workstations, and mission-critical systems where data integrity is paramount and system downtime due to memory errors is unacceptable.
  - **Overhead:** More expensive and slightly slower than non-ECC memory due to the additional circuitry and computations.

2. Virtual Memory

Virtual memory is a memory management technique that allows a computer to compensate for physical memory shortages by temporarily transferring data from RAM to disk storage. It creates the illusion that each process has its own large, contiguous address space, independent of other processes and the actual physical memory available.

2.1 Concept of Virtual Memory and Paging

- **The Problem:**
  - Physical memory (RAM) is a finite resource.
  - Multiple processes need to run concurrently, each requiring its own memory space.
  - Processes might be larger than the available physical memory.
  - Sharing memory efficiently and securely between processes is challenging.

- **Virtual Memory Solution:**
  - **Virtual Address Space:** Each process is given its own *virtual address space*, which is a logical view of memory. This space typically starts from address 0 and can be very large (e.g., 4GB for a 32-bit system, or much larger for 64-bit).
  - **Physical Address Space:** This refers to the actual addresses in RAM.
  - **Paging:** The virtual address space and the physical address space are divided into fixed-size blocks:
    - **Pages:** Blocks in the virtual address space.
    - **Frames (or Page Frames):** Blocks in the physical address space.
    - Typical page/frame sizes are 4KB, 8KB, 16KB, etc. (powers of 2).
- **How it Works (High-Level):**
  1. When a program executes, the CPU generates *virtual addresses*.
  2. A hardware component called the **Memory Management Unit (MMU)** translates these virtual addresses into *physical addresses*.
  3. This translation process is done on a page-by-page basis. A virtual page can be mapped to any available physical frame, or it might not be in physical memory at all (in which case it's on disk).
  4. If a virtual page is not in physical memory, it's loaded from disk (a "page fault").
- **Benefits of Virtual Memory:**
  - **More Programs Run Concurrently:** Allows the system to run more programs than would fit in physical memory by swapping inactive pages to disk.
  - **Memory Protection:** Each process's virtual address space is isolated, preventing one process from accidentally or maliciously accessing another's memory.
  - **Memory Sharing:** Multiple processes can share the same physical pages (e.g., for shared libraries or read-only code).
  - **Simplified Memory Management for Programmers:** Programmers don't need to worry about the physical layout of memory. They can assume a large, contiguous address space.

2.2 Page Tables, TLB (Translation Lookaside Buffer)

- **Page Tables:**
  - **Purpose:** The core data structure for virtual-to-physical address translation.
  - **Structure:** Each process has its own page table. A page table is essentially an array where each entry corresponds to a virtual page.
  - **Page Table Entry (PTE):** Each PTE contains:
    - **Frame Number:** The physical address of the page frame where the virtual page is currently stored.
    - **Valid/Invalid Bit:** Indicates whether the page is currently in physical memory (valid) or on disk (invalid).
    - **Dirty Bit (Modified Bit):** Set if the page has been written to since it was loaded into memory. Used to determine if the page needs to be written back to disk when evicted.
    - **Reference Bit (Accessed Bit):** Set when the page is accessed. Used by page replacement algorithms.
    - **Protection Bits:** Define access permissions (read, write, execute) for the page.
  - **Hierarchical Page Tables:** For very large virtual address spaces, a single, flat page table would be too large. Hierarchical page tables (e.g., two-level, multi-level) are used to reduce the memory overhead of storing page tables themselves.
- **TLB (Translation Lookaside Buffer):**
  - **The Problem:** Every memory access involving a virtual address requires at least two physical memory accesses: one to read the page table entry, and one to access the actual data. This would significantly slow down memory access.
  - **Solution:** The TLB is a small, fast, hardware cache within the MMU that stores recently used virtual-to-physical page mappings.
  - **Operation:**
    1. When the CPU generates a virtual address, the MMU first checks the TLB.
    2. **TLB Hit:** If the mapping is found in the TLB (a "TLB hit"), the physical address is quickly retrieved, and the memory access proceeds without accessing the main page table. This is extremely fast.
    3. **TLB Miss:** If the mapping is not in the TLB (a "TLB miss"), the MMU must access the main memory page table to find the mapping.

4. Once the mapping is found in the page table, it is loaded into the TLB for future fast access (potentially replacing an older entry if the TLB is full).

- **Importance:** The TLB significantly speeds up virtual memory translation, making virtual memory practical.

2.3 Page Faults

- **Definition:** A page fault (or page exception) is an event that occurs when a program tries to access a virtual page that is not currently in physical memory.
- **Page Fault Handling Process:**
  1. **Trap to OS:** The MMU detects that the "valid" bit for the accessed virtual page in the page table entry is set to "invalid." This triggers a hardware interrupt, trapping control to the operating system's page fault handler.
  2. **Determine Legitimacy:** The OS verifies if the memory access is legitimate (i.e., the process has permission to access that address) and if the page truly belongs to the process. If not legitimate, a segmentation fault or other error is generated.
  3. **Locate Page on Disk:** If legitimate, the OS locates the required page on secondary storage (e.g., the swap space on the hard drive).
  4. **Find Free Frame:** The OS finds a free page frame in physical memory.
  5. **Page Replacement (if no free frame):** If there are no free frames, the OS must select a page in memory to be "evicted" (swapped out) to make space. This involves using a page replacement algorithm (e.g., LRU, FIFO, Optimal, Clock). If the evicted page is "dirty" (modified), it must first be written back to disk.
  6. **Load Page:** The required page is read from disk into the chosen free page frame. This is an I/O operation and is relatively slow.
  7. **Update Page Table:** The OS updates the page table entry for the newly loaded page, setting the "valid" bit to "valid" and updating the frame number.
  8. **Update TLB (Optional/Hardware dependent):** The new mapping might be added to the TLB. Some architectures invalidate relevant TLB entries on page faults, forcing a fresh lookup.
  9. **Restart Instruction:** Control is returned to the instruction that caused the page fault. The instruction is then re-executed, and this time, the memory access will succeed as the page is now in memory.

- **Thrashing:**
  - **Concept:** A severe performance degradation that occurs when the operating system spends most of its time swapping pages in and out of memory rather than executing useful work.
  - **Cause:** Occurs when the total working set (the set of pages actively used by all processes) exceeds the available physical memory.
  - **Symptoms:** Very high page fault rate, extremely slow system response, high disk activity.
  - **Mitigation:** Increasing physical RAM, reducing the number of running processes, using more efficient page replacement algorithms.